

GP User Module — Implementation Lab

ADAC4CARE | Guided implementation and integration practice

Repository snapshot inspected: 10 September 2026 | Windows PowerShell

Objective: Rebuild the existing GP chart module and explain how identity, HTTP contracts, database revisions and React state combine to make a prescription change persist.

Setup / Prerequisites

This lab follows the implementation in your local ADAC4CARE project. The working GP path is React -> a same-origin Vinux server route -> Cloudflare D1. A separate Django REST Framework API exists in backend/, but the GP chart does not call it. You will run both to verify that distinction, then rebuild the working GP path. No connection between those databases is implied.

Use fictional evaluation records throughout. Values such as Test medicine and 10 mg are software fixtures, not prescribing instructions. The repository identifies its population as demo data.

Requirement	Exact evidence and use
Node.js	frontend/package.json requires >=22.13.0; the author verified Node 22.23.0.
npm	package-lock.json is present; author used npm 10.9.8. Use npm ci to respect it.
Frontend	React 19.2.6, Vinux 1.0.0-beta.5, Vite 8.0.13, TypeScript 5.9.3, Tailwind 4.2.1. App Router conventions and next/* imports are implemented by Vinux here.
Active database	D1 binding DB; Drizzle ORM declared ^0.45.2 and Drizzle Kit ^0.31.10. Drizzle defines schemas/migrations; chart routes execute prepared SQL directly.
Python	Backend Dockerfile specifies Python 3.12. The existing local virtual environment and checks used Python 3.11.9 successfully. There is no root Python version pin.
Separate backend	Django 5.2.6, DRF 3.16.1, Django ORM, pip requirements files. PostgreSQL 16 in Compose; this lab uses Django's supported SQLite URL for an isolated local exercise.
Auth	GP chart: Sites identity headers and email allowlists. Django: session authentication plus custom MFA middleware. No GP JWT login/refresh endpoint is implemented.
Tools	PowerShell 7 for -SkipHttpErrorCheck, a code editor, and a browser with Developer Tools. Node and Python must resolve in your terminal.

Open this actual workspace at C:\Users\pedim\OneDrive\Desktop\Side_projects\ADAC4CARE. The root is not a Git repository in this supplied copy. Only frontend/ has Git metadata, at commit c53e0eaccf0d9771a9babfada1fab6de8b1f35f3, with no remote configured. An exact clone URL cannot be supplied from this evidence; use the local copy. A fresh clone requires a repository URL from its owner.

Keep this document beside the supplied output/gp-lab/ files. New-GpLab.ps1, Start-Rebuild.ps1, wrangler.lab.json and Test-GpWorkflow.mjs are **new lab helpers**, not files claimed to have existed in the app. They create a separate exercise tree, preserve a reference, configure local migration tooling and verify HTTP behavior. All application excerpts below are extracted verbatim from the inspected files. Their paths are relative to the ADAC4CARE root; source line references identify the original snapshot.

You will first boot a full scaffold, then remove nine GP implementation files from that scaffold into a reference folder. Recreate those files in steps (2)-(9). For each file, type the complete reference implementation in your editor if you want writing practice; the supplied Copy-Item command is a runnable full-file checkpoint. Excerpts explain the important

parts; they are not incomplete replacement files. Shared UI primitives, fixtures, navigation and administration components remain as prerequisites because the GP page imports or displays them.

Laboratory Instructions

(0) Install, configure and boot the two services

Create an isolated learning copy so that your experiments have their own databases. Boot the implemented same-origin GP API and the separate Django process, and verify which service answers each request.

Commands / implementation. Open PowerShell terminal A. Run this initial block from any directory; keep this terminal for later rebuild commands.

```
POWERSHELL
$repo = 'C:\Users\pedim\OneDrive\Desktop\Side_projects\ADAC4CARE'
Set-Location -LiteralPath $repo
node --version
npm --version
python --version
& "$repo\output\gp-lab\New-GpLab.ps1"
$lab = Join-Path $repo 'work\gp-user-lab'
$ref = Join-Path $lab 'reference\frontend'
Set-Location -LiteralPath "$lab\frontend"
npm ci
$env:WRANGLER_SEND_METRICS = 'false'
$env:WRANGLER_LOG_PATH = '.wrangler/logs'
npx wrangler dl migrations apply DB --local --config wrangler.lab.json
```

If prompted to apply the two local migrations, enter y. A pre-existing destination causes the helper to stop without overwriting it. Use a fresh workspace copy for a repeat attempt, or resume your existing exercise rather than rerunning the setup helper.

These are the actual npm scripts in frontend/package.json:

Source: frontend/package.json (lines 8-14; verbatim excerpt).

```
JSON
{
  "scripts": {
    "dev": "vinext dev",
    "build": "vinext build",
    "start": "wrangler dev --config dist/server/wrangler.json",
    "lint": "oxlint",
    "format": "oxfmt"
  },
}
```

The helper copies the frontend environment example into the lab's .env.local. Its actual contents are:

Source: frontend/.env.example (lines 1-7; complete file).

```
DOTENV
NEXT_PUBLIC_API_URL=http://localhost:8000/api
NEXT_PUBLIC_APP_ENV=development
NEXT_PUBLIC_SITE_URL=http://localhost:3000
# Comma-separated Sites-authenticated emails granted GP prescribing permission.
# Set separately in hosted runtime settings. Local development uses the demo role.
ADAC_GP_EMAILS=
ADAC_CARER_EMAILS=
```

Open terminal B, reserved for the frontend server:

POWERSHELL

```
Set-Location -LiteralPath 'C:\Users\pedim\OneDrive\Desktop\Side_projects\ADAC4CARE\work\gp-user-lab\frontend'
npm run dev -- --port 3000
```

Open terminal C, reserved for Django:

POWERSHELL

```
Set-Location -LiteralPath 'C:\Users\pedim\OneDrive\Desktop\Side_projects\ADAC4CARE\work\gp-user-lab\backend'
python -m venv .venv
& .\venv\Scripts\python.exe -m pip install -r requirements/dev.txt
$env:DJANGO_SETTINGS_MODULE = 'config.settings.dev'
$env:DJANGO_DEBUG = 'true'
$env:DATABASE_URL = 'sqlite:///gp-lab.sqlite3'
& .\venv\Scripts\python.exe manage.py migrate --noinput
& .\venv\Scripts\python.exe manage.py runserver 127.0.0.1:8000 --noreload
```

In terminal A, inspect both HTTP responses:

POWERSHELL

```
$chartUrl = 'http://localhost:3000/api/prescriber-chart?home=banksia-house&client=james-miller'
Invoke-RestMethod -Uri $chartUrl | ConvertTo-Json -Depth 8
$django = Invoke-WebRequest 'http://localhost:8000/api/clients/' -SkipHttpErrorCheck
$django.StatusCode
$django.Content
```

Key lines. The scaffold excludes existing .env secrets, installed dependencies, Git history and database state. `npm ci` installs the lockfile; Python's venv keeps pip packages isolated. The helper's Wrangler config uses the exact binding name and local placeholder database ID from frontend/vite.config.ts. Both tools use `.wrangler/state/v3` under the lab frontend. Applying only Django migrations cannot create D1 tables.

DATABASE_URL overrides the copied backend example's PostgreSQL URL for this terminal. Set DJANGO_DEBUG=true before importing settings: `base.py` computes secure-cookie and HTTPS-redirection settings before `dev.py` assigns DEBUG=True. A terminal environment disappears when that terminal closes; repeat the three backend environment assignments when restarting it. No virtual-environment activation is required because each command uses its Python executable directly.

Contract. A GP GET on port 3000 returns HTTP 200 and `{records:[], administrations:[], canPrescribe:true, canAdminister:true}` in a fresh local lab. Boolean property order is irrelevant. Permission flags describe local evaluator capabilities; the selected UI role further restricts actions. The unauthenticated Django GET returns HTTP 403 and `{"detail":"Authentication credentials were not provided."}`. A 403 proves the protected Django endpoint is reachable; it is not a successful authenticated data read.

Why / failure modes / pattern. This is a **same-origin server API with runtime database bindings** plus a **separate service boundary**. The browser's relative `/api/prescriber-chart` goes to its own origin. `NEXT_PUBLIC_API_URL` is only used in a different, currently uncalled helper. Missing D1 migrations yield the chart's 503 load error; a wrong port gives connection failure; a wrong home/client gives 404. No amount of starting Django makes the GP chart read Django records.

Expected result. Terminal B prints a local frontend URL. Terminal C completes migrations and starts the development server. The GP GET is 200, while the unauthenticated Django read is 403. Use `localhost` exactly for the frontend; the author observed its printed `localhost` URL working where a numeric-loopback URL did not.

Verification before continuing. Open `http://localhost:3000/people`, select **General practitioner** under **Demo access level**, keep **Banksia House**, and open James Miller's chart. In Developer Tools (F12), choose Network, reload, filter `prescriber-chart`, and confirm the request uses port 3000 with status 200. Reserve `regular-slot-8` for the automated exercise in step (14). If port 3000 or 8000 is occupied, use another port consistently in every lab URL; the author used 3100 and 8100.

(1) Map the integration and prepare the rebuild

Identify which files own presentation, state, transport, rules and storage. Move only the nine GP files in the isolated scaffold into its reference directory so you can rebuild them in dependency order.

Commands / implementation. Stop terminal B's frontend with Ctrl+C; leave terminal A open. In A:

```
POWERSHELL
Set-Location -LiteralPath $repo
Get-ChildItem -LiteralPath "$lab\frontend\app\api" -Directory
Get-Content -LiteralPath "$lab\frontend\docs\gp-medication-chart.md"
& "$repo\output\gp-lab\Start-Rebuild.ps1"
Get-ChildItem -LiteralPath $ref -Recurse -File | Select-Object FullName
```

Key lines. Start-Rebuild.ps1 validates every move stays inside work/gp-user-lab; the destination is reference/frontend, outside the frontend TypeScript project. It refuses to replace an existing reference. The original application stays intact. The missing modules deliberately prevent the GP page from compiling until step (9); pure-function checks are introduced earlier where dependencies permit them.

```
TEXT
RoleProvider + demo home/client JSON
|
MedicationChartsContent -> GpMedicationTable -> MedicineRows
|                                     |
+---- usePrescriberChart <-----+
|
GET / PATCH /api/prescriber-chart
|
chartAccess -> validation -> prepared SQL -> D1 DB
|
acknowledged record/version -> React state -> UI

Django /api/medication-orders/ -> Django ORM -> separate DB
(no call from the GP chart)
```

Contract. The page selects home/client identifiers; the hook returns records keyed by medication ID and save/reload functions; the route consumes JSON and returns the acknowledged version. Chart capabilities arrive with the read response. The shared carer endpoint is only needed to preserve the GP's read-only view of administration history and the table's imports.

Why / failure modes / pattern. This is **separation of concerns** with a **custom-hook data access layer**. It is not a controller-service-repository architecture: the active route contains orchestration and SQL itself. If the hook sends to Django, its payload no longer matches. If a row uses an index as its permanent identity, updates can target the wrong medicine. If the same React instance survives a patient change with old state, another patient's draft can appear.

Expected result. Nine reference files exist: four lib files, one hook, two components, and two route/page files. Shared fixtures and primitives remain in the scaffold.

Verification before continuing. Without consulting the diagram, trace a Dose blur event to the eventual database insert and back to the displayed Saved notice. Record one responsible file for each transition. Do not restart the frontend yet.

(2) Recreate the prescription data contract and pure rules

Define the exact object both sides exchange before wiring HTTP. Rebuild `lib/prescriber-chart.ts`; its pure functions give the browser early feedback and the server independent validation.

Commands / implementation. In terminal A:

```
POWERSHELL
Set-Location -LiteralPath "$lab\frontend"
Copy-Item -LiteralPath "$ref\lib\prescriber-chart.ts" -Destination '.\lib\prescriber-chart.ts'
Get-Content -LiteralPath '.\lib\prescriber-chart.ts'
```

Recreate the whole file from that reference. Its contract and initialization are:

Source: `frontend/lib/prescriber-chart.ts` (lines 1-33; verbatim excerpt).

```
TYPESCRIPT
import type { Medication } from '../data/demo/types';

export type Signature = { strokes: number[][][]; initials?: string; signedAt: string; signedBy: string };
export type Prescription = {
  name: string;
  dose: string;
  route: string;
  frequency: string;
  start_date: string;
  end_date: string;
  refer_prn: boolean;
  times: string[];
  instructions: string;
  max_dose: string;
  prescriber_signature: Signature | null;
};
export type ChartRecord = { medicationId: string; version: number; prescription: Prescription };
export type AdministrationRecord = { medicationId: string; cellKey: string; orderVersion: number; date: string;
  time: string; qty: string; signature: Signature };
export type AdministrationInput = { medicationId: string; cellKey: string; orderVersion: number; strokes:
  number[][][]; initials?: string; confirmed: boolean };
export type ChartResponse = { records: ChartRecord[]; administrations: AdministrationRecord[]; canPrescribe:
  boolean; canAdminister: boolean };
export const prescriptionFields = ['name', 'dose', 'route', 'frequency', 'start_date', 'end_date', 'instructions',
  'max_dose'] as const;
export type PrescriptionField = (typeof prescriptionFields)[number];
export type SignatureField = 'prescriber_signature';

export function initialPrescription(medication?: Medication): Prescription {
  return {
    name: medication?.name ?? '', dose: medication?.dose ?? '', route: medication?.route ?? '',
    frequency: medication?.schedule ?? '', start_date: medication?.prescribedDate ?? '', end_date: '',
    refer_prn: medication?.type === 'prn', times: Array.from({ length: 6 }, (_, index) => medication?.times[index]
      ?? ''),
    instructions: medication?.instructions ?? '', max_dose: medication?.maxDailyDose ?? '',
    prescriber_signature: null,
  };
}
```

Its amendment function is:

Source: `frontend/lib/prescriber-chart.ts` (lines 63-66; verbatim excerpt).

```
TYPESCRIPT
export function amendPrescription<K extends PrescriptionField | 'refer_prn' | 'times'>(p: Prescription, field: K,
  value: Prescription[K]): Prescription {
  if (p[field] === value) return p;
  return { ...p, [field]: value, prescriber_signature: null };
}
```

The same file supplies the runtime validator called on both sides of the HTTP boundary:

Source: frontend/lib/prescriber-chart.ts (lines 39-60; verbatim excerpt).

TYPESCRIPT

```
export function validatePrescription(value: unknown): asserts value is Prescription {
  if (!value || typeof value !== 'object' || Array.isArray(value)) throw new Error('Invalid prescription.');
```

```
  const p = value as Prescription;
  if (Object.keys(p).some((key) => ![...prescriptionFields, 'prescriber_signature', 'refer_prn',
    'times'].includes(key))) throw new Error('Only prescriber fields can be edited.');
```

```
  for (const field of prescriptionFields) {
    if (typeof p[field] !== 'string' || p[field].length > (field === 'instructions' ? 2000 : 200)) throw new
      Error('Invalid prescriber field or text length.');
```

```
  }
  if (typeof p.refer_prn !== 'boolean') throw new Error('Refer PRN must be ticked or unticked.');
```

```
  if (!Array.isArray(p.times) || p.times.length !== 6 || p.times.some((time) => typeof time !== 'string' || (time
    !== '' && !/^[01]\d|2[0-3]):[0-5]\d$/.test(time)))) throw new Error('Enter up to six valid administration
    times.');
```

```
  const times = p.times.filter(Boolean);
  if (new Set(times).size !== times.length) throw new Error('Administration times must be unique.');
```

```
  if (!p.name.trim()) throw new Error('Enter a regular medicine name.');
```

```
  if (p.start_date && !validDate(p.start_date)) throw new Error('Enter a valid start date.');
```

```
  if (p.end_date && !validDate(p.end_date)) throw new Error('Enter a valid stop date.');
```

```
  if (p.end_date && (!p.start_date || p.end_date < p.start_date)) throw new Error('Stop date must be on or after
    the start date.');
```

```
  for (const field of ['prescriber_signature'] as const) {
    const signature = p[field];
    if (signature === null) continue;
    validateStrokes(signature?.strokes);
    if (!p.dose.trim() || !p.route.trim() || !p.frequency.trim() || !p.start_date) throw new Error('Complete
      medicine, dose, route, frequency and start date before signing.');
```

```
  }
}
```

Key lines. `initialPrescription()` creates all required fields, including six time slots and a nullable signature. Existing fixture medicines start with version 0 until a revision is persisted. The validator accepts eight text fields in `prescriptionFields`, `refer_prn`, `times`, and `prescriber_signature`; an old test title saying "seven" is stale. name must be nonblank even for an unsigned save. Signing additionally needs dose, route, frequency and start date. Instructions allow 2,000 characters; other text fields allow 200.

Contract. A PATCH contains `medicationId`, integer version `>= 0`, and the **whole** `Prescription`, not just `{dose: ...}`. This route's PATCH semantics append a complete snapshot. Dates are empty strings or valid YYYY-MM-DD; stop date requires a start date and cannot precede it. `times` must contain exactly six strings, each blank or a distinct valid 24-hour time. Unknown keys are rejected. Validation errors become HTTP 400 `{error: string}` at the route.

Why / failure modes / pattern. These are a **shared data transfer object (DTO)** and **boundary validation**, with **signature invalidation on amendment**. TypeScript types disappear at runtime, so a type assertion cannot protect against malformed network input. Requiring a complete shape keeps each revision self-contained. Carrying the old signature into a changed order would incorrectly associate an earlier approval with new values; `amendPrescription` clears it.

Expected result. The source file compiles as ordinary TypeScript. A modified dose yields a new object with a null signature; an unchanged scalar returns the original object. A newly allocated `times` array can invalidate the signature even if its contents match, because this helper first uses reference equality.

Verification before continuing. Compare your file's hash to the reference:

POWERSHELL

```
(Get-FileHash '.\lib\prescriber-chart.ts').Hash -eq (Get-FileHash "$ref\lib\prescriber-chart.ts").Hash
```

Expected: True for the exact checkpoint. Explain why dose cannot be renamed to dosage in only the form. Automated domain tests will run after the permission helper is restored in step (4).

(3) Build the persisted revision model

Inspect the schema and execute the existing migrations against the lab's D1 database. This makes clear which values are relational keys and which values are serialized inside each prescription snapshot.

Commands / implementation. The schema is a retained shared prerequisite, already copied in step (0). Its GP tables are:

Source: frontend/db/schema.ts (lines 1-20; verbatim excerpt).

TYPESCRIPT

```
import { index, integer, primaryKey, sqliteTable, text } from 'drizzle-orm/sqlite-core';

// Append-only versions retain the prescription and signature that were authorised.
export const prescriptionRevisions = sqliteTable('prescription_revisions', {
  homeId: text('home_id').notNull(),
  clientId: text('client_id').notNull(),
  medicationId: text('medication_id').notNull(),
  version: integer('version').notNull(),
  prescription: text('prescription').notNull(),
  actor: text('actor').notNull(),
  savedAt: text('saved_at').notNull(),
}, (table) => [primaryKey({ columns: [table.homeId, table.clientId, table.medicationId, table.version] })]);

export const administrationSignatures = sqliteTable('administration_signatures', {
  homeId: text('home_id').notNull(), clientId: text('client_id').notNull(),
  medicationId: text('medication_id').notNull(), cellKey: text('cell_key').notNull(),
  orderVersion: integer('order_version').notNull(),
  date: text('date').notNull(), time: text('time').notNull(), qty: text('qty').notNull(),
  signature: text('signature').notNull(), actor: text('actor').notNull(), savedAt: text('saved_at').notNull(),
}, (table) => [primaryKey({ columns: [table.homeId, table.clientId, table.medicationId, table.cellKey] })]);
```

POWERSHELL

```
Get-Content -LiteralPath '.\drizzle\0001_short_random.sql'
npx wrangler d1 migrations apply DB --local --config wrangler.lab.json
npx wrangler d1 execute DB --local --config wrangler.lab.json --command "SELECT name FROM sqlite_master WHERE
  type='table' ORDER BY name"
```

Key lines. The prescription primary key is (home_id, client_id, medication_id, version). A medicine has many immutable-through-the-API versions. prescription stores JSON text; it is not a foreign key to a medications table. Demo homes/clients/medications live in imported JSON and have no matching D1 parent tables. actor and saved_at are separate metadata. The administration primary key identifies a signed cell rather than an entire prescription.

Contract. Database snake_case columns are mapped into camelCase API properties such as medicationId. Within Prescription, fields such as start_date deliberately remain snake_case. The insert must use the same three partition keys as the latest-version query. order_version associates an administration with the prescription version visible when signed.

Why / failure modes / pattern. This is an **append-only revision history** with a **composite partition key**. It preserves old signed snapshots without representing all domain changes as a full event-sourcing system. Omitting home/client from the SQL predicate can mix records. Updating the latest row in place destroys history. The schema alone does not forbid SQL UPDATE/DELETE by a privileged database operator; append-only behavior is enforced by this API, not a database trigger.

Expected result. The second migration application reports no remaining work. The table listing includes prescription_revisions, administration_signatures, medication_records, and migration bookkeeping. No prescriptions should have been added by merely opening a chart.

Verification before continuing. Run:

POWERSHELL

```
npx wrangler d1 execute DB --local --config wrangler.lab.json --command "SELECT COUNT(*) AS revisions FROM
  prescription_revisions"
```

Expected: 0 if you followed the no-edit baseline. Explain why imported demo medicine names can appear even though this count is zero. Do not regenerate migrations: `drizzle.config.ts` describes generation, whereas applying the checked-in SQL changes the actual local database.

(4) Rebuild identity, resource context and database bindings

Recreate the small server helpers that decide who may act, which client is being addressed and which database is used. These are prerequisites for the route; a role dropdown is insufficient to provide server authorization.

Commands / implementation. In terminal A:

POWERSHELL

```
foreach ($file in 'chart-access.ts','chart-db.ts','chart-server.ts') {
    Copy-Item -LiteralPath (Join-Path "$ref\lib" $file) -Destination '.\lib'
}
```

The complete authorization helper is:

Source: `frontend/lib/chart-access.ts` (lines 1-16; complete file).

TYPESCRIPT

```
// The demo role selector is never sufficient to grant hosted write access.
export function chartAccess(headers: Headers, allowedEmails: string, localDevelopment: boolean, carerEmails = '')
{
    const email = headers.get('oai-authenticated-user-email')?.toLowerCase() ?? '';
    const id = headers.get('oai-authenticated-user-id');
    const authenticated = localDevelopment || Boolean(id && email);
    const gp = localDevelopment || allowedEmails.split(',').map((item) =>
        item.trim().toLowerCase()).filter(Boolean).includes(email);
    const carer = localDevelopment || carerEmails.split(',').map((item) =>
        item.trim().toLowerCase()).filter(Boolean).includes(email);
    return {
        authenticated,
        canPrescribe: authenticated && gp,
        actor: localDevelopment ? (headers.get('x-chart-role') === 'carer' ? 'Local carer evaluation' : 'Local GP
        evaluation') : id ?? '',
        canWrite: authenticated && gp && headers.get('x-chart-role') === 'gp',
        canAdminister: authenticated && carer,
        canSign: authenticated && carer && headers.get('x-chart-role') === 'carer',
    };
}
```

The complete database and context helpers are:

Source: `frontend/lib/chart-db.ts` (lines 1-6; complete file).

TYPESCRIPT

```
import { env } from 'cloudflare:workers';

type ChartEnv = { DB: D1Database; ADAC_GP_EMAILS?: string; ADAC_CARER_EMAILS?: string };
export function chartDatabase() { return (env as unknown as ChartEnv).DB; }
export function gpEmails() { return (env as unknown as ChartEnv).ADAC_GP_EMAILS ?? ''; }
export function carerEmails() { return (env as unknown as ChartEnv).ADAC_CARER_EMAILS ?? ''; }
```

Source: `frontend/lib/chart-server.ts` (lines 1-7; complete file).

TYPESCRIPT

```
import { groupHomes } from '@data/demo';
export function chartContext(url: URL) {
    const home = groupHomes.find((item) => item.id === url.searchParams.get('home'));
    const client = home?.clients.find((item) => item.id === url.searchParams.get('client'));
    return home && client ? { home, client } : null;
}
```

TYPESCRIPT / CONTINUED

```
export const chartJson = (body: unknown, status = 200) => Response.json(body, { status, headers: {
  'Cache-Control': 'no-store' } });
```

Key lines. Hosted identity comes from `oai-authenticated-user-id` and `oai-authenticated-user-email`.

`ADAC_GP_EMAILS` is a comma-separated server-runtime allowlist; empty denies hosted GP writes. `canPrescribe` describes an identity's capability, while `canWrite` also requires `x-chart-role: gp`. `chartContext` validates the home/client pair against the demo population; unlike UI fallback helpers, it returns null for an invalid pair. `chartJson` prevents caching of route responses.

Contract. The browser sends no bearer token and does not manufacture the Sites identity headers. The hosted edge must authenticate the viewer and supply trusted headers; header presence alone is not cryptographic verification inside this code. A local `import.meta.env.DEV` argument bypasses identity allowlists for evaluation, but still requires the matching role header on writes. `DB` is a runtime object imported from `cloudflare:workers`, not a public string database URL.

Why / failure modes / pattern. This is **trusted-proxy identity propagation** plus **server-side capability enforcement**. The deployment must prevent untrusted clients from bypassing the authenticating edge or spoofing its headers. That infrastructure guarantee was not verified against a live hosted deployment. Putting an allowlist in browser state would let the client decide its own authority. Reading `DB` from `process.env` would yield no usable database object. Misspelling a client ID must not silently select another patient.

Expected result. Pure tests can now import both domain and permission helpers. Hosted-mode tests reject a role header alone and deny empty allowlists, even though local UI evaluation permits the selected GP.

Verification before continuing. Run the repository's exact test command:

POWERSHELL

```
node --experimental-strip-types --test tests/prescriber-chart.test.ts
```

Expected: 10 tests pass. The tests call `chartAccess(..., false)` to exercise hosted rules without changing production identities. Explain why setting the local dropdown to GP cannot establish a real user's identity. No role-provisioning or hosted sign-in flow is being implemented in this lab.

(5) Rebuild the chart read endpoint

Create the complete `app/api/prescriber-chart/route.ts` module from the reference. Study GET first: it turns database revisions and administration rows into the precise read model the hook needs.

Commands / implementation. The checkpoint restores the whole route file, including PATCH studied next:

POWERSHELL

```
Copy-Item -LiteralPath "$ref\app\api\prescriber-chart\route.ts" -Destination '.\app\api\prescriber-chart\route.ts'
```

These are its imports, route setting, access wrapper and GET handler:

Source: `frontend/app/api/prescriber-chart/route.ts` (lines 1-28; verbatim excerpt).

TYPESCRIPT

```
import { chartDatabase, gpEmails, carerEmails } from '@lib/chart-db';
import { chartAccess } from '@lib/chart-access';
import { initialPrescription, prescriptionChanged, validatePrescription, type Prescription } from
  '@lib/prescriber-chart';
import { chartContext as context, chartJson as json } from '@lib/chart-server';

export const dynamic = 'force-dynamic';
const access = (request: Request) => chartAccess(request.headers, gpEmails(), import.meta.env.DEV, carerEmails());

export async function GET(request: Request) {
```

TYPESCRIPT / CONTINUED

```

const user = access(request);
if (!user.authenticated) return json({ error: 'Sign in to view this medication chart.' }, 401);
const ctx = context(new URL(request.url));
if (!ctx) return json({ error: 'Client not found.' }, 404);
try {
  const { results } = await chartDatabase().prepare(`SELECT medication_id, version, prescription FROM
    prescription_revisions
    WHERE home_id = ? AND client_id = ? AND (medication_id, version) IN
    (SELECT medication_id, MAX(version) FROM prescription_revisions WHERE home_id = ? AND client_id = ? GROUP BY
      medication_id)`);
  .bind(ctx.home.id, ctx.client.id, ctx.home.id, ctx.client.id).all<{ medication_id: string; version: number;
    prescription: string }>();
  const administrationResult = await chartDatabase().prepare('SELECT medication_id, cell_key, order_version,
    date, time, qty, signature FROM administration_signatures WHERE home_id = ? AND client_id = ?')
    .bind(ctx.home.id, ctx.client.id).all<{ medication_id: string; cell_key: string; order_version: number;
      date: string; time: string; qty: string; signature: string }>();
  return json({ canPrescribe: user.canPrescribe, canAdminister: user.canAdminister,
    records: results.map((r) => ({ medicationId: r.medication_id, version: r.version, prescription: {
      ...initialPrescription(ctx.client.medications.find((m) => m.id === r.medication_id)),
      ...JSON.parse(r.prescription) } })),
    administrations: administrationResult.results.map((r) => ({ medicationId: r.medication_id, cellKey:
      r.cell_key, orderVersion: r.order_version, date: r.date, time: r.time, qty: r.qty, signature:
      JSON.parse(r.signature) })),
  });
} catch {
  return json({ error: 'The chart could not be loaded. Please retry.' }, 503);
}
}

```

Key lines. The outer SELECT restricts home/client; the subquery finds each medication's MAX(version) in that same partition. .bind(...) supplies parameters without interpolating values into SQL. The response overlays saved JSON on initialPrescription so missing older optional fields receive defaults. A second query loads administration signatures, deserializing their JSON separately.

Contract. GET requires ?home=banksia-house&client=james-miller; no request body. Success is 200 with records, administrations, canPrescribe, canAdminister. Records only include medicines with saved revisions: the UI supplies unedited fixture orders. Errors are 401 unauthenticated, 404 invalid context, or 503 for caught database/load failures, each {error: string} with Cache-Control: no-store.

Why / failure modes / pattern. This is a **read model projection** from **versioned persistence**, with a **parameterized query**. The UI needs the newest row once, not every historical version. Returning the entire SQL result object instead of records would break the hook's iteration. Failing to JSON.parse the stored prescription would give the UI a string. The partition key separates records; it does not implement per-client access grants: any authenticated hosted viewer can read a valid demo context under current rules.

Expected result. The restored module exports GET and PATCH, but the page remains incomplete until step (9). The previously observed fresh GET body is now explainable from these statements rather than from assumptions about a patient API.

Verification before continuing. In the restored file, locate every home_id = ? AND client_id = ? condition and pair its placeholders with .bind. Predict the difference between an existing medicine with no revision and an invalid client: the first is supplied by fixtures in the UI, while the second is a 404. Runtime checks resume in step (9).

(6) Implement the write contract, concurrency check and signature attribution

Work through PATCH in the restored route. This is the integration core: authorization, a complete DTO, validation, version comparison and one conditional insert must agree before the UI may claim success.

Commands / implementation. Recreate this complete handler in its original file if writing it by hand; preserve the imports, GET and access wrapper from step (5).

Source: frontend/app/api/prescriber-chart/route.ts (lines 30-81; verbatim excerpt).

TYPESCRIPT

```
export async function PATCH(request: Request) {
  const user = access(request);
  if (!user.canWrite) return json({ error: 'Only an authorised GP can edit the prescriber section.' }, 403);
  const origin = request.headers.get('origin');
  if (!origin || origin !== new URL(request.url).origin) return json({ error: 'Invalid request origin.' }, 403);
  const ctx = context(new URL(request.url));
  if (!ctx) return json({ error: 'Client not found.' }, 404);
  let body;
  try {
    const raw = await request.text();
    if (raw.length > 500_000) return json({ error: 'Signature is too large.' }, 413);
    body = JSON.parse(raw);
    if (!body || Object.keys(body).some((key) => ![ 'medicationId', 'version', 'prescription' ].includes(key)))
      throw new Error('Only prescriber fields can be edited.');
```

if (typeof body.medicationId !== 'string' || !Number.isInteger(body.version) || body.version < 0) throw new Error('Invalid chart row.');

validatePrescription(body.prescription);

```
  } catch (error) {
    return json({ error: error instanceof Error ? error.message : 'Invalid prescription.' }, 400);
  }

  const medication = ctx.client.medications.find((item) => item.id === body.medicationId);
  const newSlot = /^(regular|prn)-slot-[1-8]$/.test(body.medicationId);
  if (!medication && !newSlot) return json({ error: 'Medicine row not found.' }, 404);
  if (newSlot) {
    const type = body.medicationId.startsWith('prn-') ? 'prn' : 'routine';
    const occupied = ctx.client.medications.filter((item) => item.type === type).length;
    if (Number(body.medicationId.slice(-1)) <= occupied) return json({ error: 'This chart row already belongs to an existing medicine.' }, 400);
  }

  try {
    const db = chartDatabase();
    const previous = await db.prepare('SELECT version, prescription FROM prescription_revisions WHERE home_id = ?
      AND client_id = ? AND medication_id = ? ORDER BY version DESC LIMIT 1')
      .bind(ctx.home.id, ctx.client.id, body.medicationId).first<{ version: number; prescription: string }>();
    if ((previous?.version ?? 0) !== body.version) return json({ error: 'This row changed in another session.
      Reload the chart before editing again.' }, 409);
    const before: Prescription = { ...initialPrescription(medication), ...(previous ?
      JSON.parse(previous.prescription) : {}) };
    const next = body.prescription as Prescription;
    const changed = prescriptionChanged(before, next);
    const now = new Date().toISOString();
    for (const field of ['prescriber_signature'] as const) {
      if (next[field]) {
        const sameDrawing = JSON.stringify(next[field]?.strokes) === JSON.stringify(before[field]?.strokes);
        // An old drawing may never be carried onto changed prescription details.
        if (changed && sameDrawing) return json({ error: 'Prescription details changed. Please sign again.' },
          400);
        next[field] = !changed && sameDrawing ? before[field] : { strokes: next[field]!.strokes, signedAt: now,
          signedBy: user.actor };
      }
    }
    const result = await db.prepare(`INSERT INTO prescription_revisions (home_id, client_id, medication_id,
      version, prescription, actor, saved_at)
      SELECT ?, ?, ?, ?, ?, ?, ? WHERE COALESCE((SELECT MAX(version) FROM prescription_revisions WHERE home_id = ?
      AND client_id = ? AND medication_id = ?), 0) = ?`)
      .bind(ctx.home.id, ctx.client.id, body.medicationId, body.version + 1, JSON.stringify(next), user.actor,
        now, ctx.home.id, ctx.client.id, body.medicationId, body.version).run();
    if (!result.meta.changes) return json({ error: 'This row changed in another session. Reload the chart before
      editing again.' }, 409);
```

TYPESCRIPT / CONTINUED

```

    return json({ medicationId: body.medicationId, version: body.version + 1, prescription: next });
  } catch {
    return json({ error: 'Your changes were not saved. Please retry.' }, 503);
  }
}

```

Key lines. Access and Origin checks precede parsing. The top-level body allowlist rejects attempts to submit actor or unrelated fields. `request.text().length` limits the parsed string to 500,000 JavaScript code units; this is not a streaming byte limit. Existing medication IDs and unoccupied `regular-slot-1` through `regular-slot-8` / `prn-slot-1` through `prn-slot-8` are accepted. Slot indices at or below the fixture count for that type are rejected.

The first version read gives an intelligible early conflict. The `INSERT ... SELECT ... WHERE COALESCE(MAX(version), 0) = ?` checks the version again atomically at the write. Checking only before the insert leaves a race in which two clients both read the same old version. The composite primary key is an additional uniqueness constraint. Unexpected database exceptions still fall through to 503, so not every conceivable storage conflict is guaranteed to surface as 409.

Contract. Send `Content-Type: application/json`, `X-Chart-Role: gp`, a same-origin Origin, and `{medicationId, version, prescription}`. The browser supplies Origin automatically for this write; the lab HTTP client sets it explicitly. Success is **200**, including for a new slot: `{medicationId, version: old+1, prescription}`. The server stamps a new drawing with its own actor and UTC timestamp. The normal error envelope is `{error: string}: 400 malformed/invalid fields or reused signature on changed details, 403 role/origin denial, 404 context/row not found, 409 stale version, 413 oversize body, 503 storage failure.`

Why / failure modes / pattern. This combines **optimistic concurrency control (compare-and-swap)**, **server-authoritative attribution**, and **append-only snapshots**. "Optimistic" here concerns database concurrency; it does not mean the UI reports success before saving. Retrying the same PATCH after a successful but lost response normally returns 409 because its expected version is stale. The prescription endpoint has no idempotency key. Read current state to resolve uncertainty; do not blindly increment the version and resend an old draft.

The signature branch prevents carrying identical old strokes onto changed details. It cannot prove that a person drew a fresh signature at that instant. It compares stored data and uses trusted server identity; it is not a certificate-backed digital signature or proof of clinical approval.

Expected result. Saving version 0 creates version 1. Submitting version 0 again returns 409 without another row. Modifying a signed dose through the UI sends a null signature; re-signing produces a new revision while the old signed JSON remains.

Verification before continuing. Run the existing isolated route/SQL harness:

POWERSHELL

```
node tests/chart-administrations.mjs
```

Expected: a `Passed production-role permissions...` message. It executes real route SQL using an in-memory SQLite adapter and forces production-mode access logic. It is not a live edge-authentication test and does not exercise the real `chartContext` lookup. Step (14) adds real local HTTP checks, including stale prescription writes and signature invalidation.

(7) Connect HTTP responses to React state

Rebuild the hook that loads chart data and sends saves. It translates network responses into state the table can render, and it prevents older reads from simply replacing newer acknowledged records.

Commands / implementation. Recreate the complete file, including the shared administration method, using the reference checkpoint:

POWERSHELL

```
Copy-Item -LiteralPath "$ref\hooks\use-prescriber-chart.ts" -Destination '.\hooks\use-prescriber-chart.ts'
```

The complete hook is:

Source: frontend/hooks/use-prescriber-chart.ts (lines 1-77; complete file).

TYPESCRIPT

```
'use client';

import { useCallback, useEffect, useRef, useState } from 'react';
import type { AdministrationInput, AdministrationRecord, ChartRecord, ChartResponse, Prescription } from
  '@/lib/prescriber-chart';
import type { UserRole } from '@/data/demo/types';

export function usePrescriberChart(home: string, client: string, role: UserRole) {
  const [records, setRecords] = useState<Record<string, ChartRecord>>({});
  const [loaded, setLoaded] = useState(false);
  const [allowed, setAllowed] = useState(false);
  const [canSign, setCanSign] = useState(false);
  const [administrations, setAdministrations] = useState<AdministrationRecord[]>([]);
  const [error, setError] = useState('');
  const recordsRef = useRef(records);
  const saving = useRef(false);
  const endpoint = `/api/prescriber-chart?home=${encodeURIComponent(home)}&client=${encodeURIComponent(client)}`;
  const reload = useCallback(async () => {
    if (saving.current) return;
    try {
      const response = await fetch(endpoint, { cache: 'no-store' });
      const data = await response.json() as ChartResponse & { error?: string };
      if (!response.ok) throw new Error(data.error);
      // Do not overwrite a newer save with a read that started before it.
      if (saving.current) return;
      const result = data as ChartResponse;
      setRecords((previous) => {
        const next = { ...previous };
        for (const record of result.records) if ((next[record.medicationId]?.version ?? 0) <= record.version)
          next[record.medicationId] = record;
        recordsRef.current = next;
        return next;
      });
      setAllowed(result.canPrescribe);
      setCanSign(result.canAdminister);
      setAdministrations((previous) => {
        const merged = new Map(previous.map((record) => [`${record.medicationId}:${record.cellKey}`, record]));
        for (const record of result.administrations) merged.set(`${record.medicationId}:${record.cellKey}`,
          record);
        return [...merged.values()];
      });
      setLoaded(true);
      setError('');
    } catch (e) { setError(e instanceof Error ? e.message : 'Unable to load saved prescriptions. '); }
  }, [endpoint]);
  useEffect(() => {
    void reload();
    const timer = window.setInterval(() => { if (document.visibilityState === 'visible') void reload(); }, 15000);
    window.addEventListener('focus', reload);
    return () => { window.clearInterval(timer); window.removeEventListener('focus', reload); };
  }, [reload]);

  const save = async (medicationId: string, prescription: Prescription, expectedVersion: number) => {
```

TYPESCRIPT / CONTINUED

```

    if (!loaded || !allowed || role !== 'gp') throw new Error('Only an authorised GP can edit this section.');
```

```

    if (saving.current) throw new Error('Please wait for the current save to finish.');
```

```

    saving.current = true;
```

```

    try {
```

```

      const response = await fetch(endpoint, {
```

```

        method: 'PATCH', headers: { 'Content-Type': 'application/json', 'X-Chart-Role': role },
```

```

        body: JSON.stringify({ medicationId, prescription, version: expectedVersion }),
```

```

      });
```

```

      const result = await response.json() as ChartRecord & { error?: string };
```

```

      if (!response.ok) throw new Error(result.error ?? 'Changes were not saved.');
```

```

      const record = result as ChartRecord;
```

```

      recordsRef.current = { ...recordsRef.current, [medicationId]: record };
```

```

      setRecords(recordsRef.current);
```

```

      return record;
```

```

    } finally { saving.current = false; }
```

```

  });
```

```

  const signAdministration = async (input: AdministrationInput) => {
```

```

    if (!loaded || !canSign || role !== 'carer') throw new Error('Only an authorised carer can sign this cell.');
```

```

    const response = await fetch(endpoint.replace('/prescriber-chart?', '/chart-administrations?'), {
```

```

      method: 'POST', headers: { 'Content-Type': 'application/json', 'X-Chart-Role': role }, body:
```

```

      JSON.stringify(input),
```

```

    });
```

```

    const result = await response.json() as AdministrationRecord & { error?: string };
```

```

    if (!response.ok) throw new Error(result.error ?? 'Signature was not saved.');
```

```

    setAdministrations((previous) => [...previous.filter((r) => r.medicationId !== result.medicationId ||
```

```

      r.cellKey !== result.cellKey), result]);
```

```

  };
  return { records, administrations, loaded, error, reload, save, signAdministration, canPrescribe: loaded &&
    allowed && role === 'gp', canAdminister: loaded && canSign && role === 'carer' };
}
```

Key lines. endpoint URL-encodes the selected home and client. reload checks response.ok; HTTP 400/403/409 do not cause fetch itself to reject. Records are indexed by medicationId for direct row lookup. A version comparison prevents a lower-version response from overwriting a higher one. The hook blocks another save while saving.current is true, and resumes in finally even when a request fails.

recordsRef holds the latest acknowledged map for asynchronous callbacks; setRecords causes React to render. They are related but not interchangeable: writing a ref alone does not update the screen. A row is only inserted into this shared map after the server returns success. The hook neither writes prescriptions to localStorage nor queues offline mutations.

Contract. GET consumes ChartResponse; PATCH returns one ChartRecord. A save sends the row's expected version. save rejects its promise with the server's error, allowing the row to show a retry message. The return value canPrescribe is the intersection of successful load, server capability and role === 'gp'. Browser fetch uses same-origin credentials by default; this code adds no JWT or explicit bearer header.

Why / failure modes / pattern. This is a **custom hook as an API adapter, server-acknowledged state reconciliation, and polling/focus revalidation**. Replacing server truth with a locally incremented version would conceal rejected writes. Ignoring response.ok would let an error object masquerade as a prescription. The code assumes route responses are JSON; a plain-text runtime 403 can instead surface as a JSON parsing error, observed during author verification. A future robust transport wrapper would check content type or preserve raw error text.

The hook refreshes every 15 seconds while visible and on focus. That is eventual refresh, not WebSockets. It skips reads around saves and compares record versions, but is not a fully ordered response system for every field. For example, permission flags have no response sequence number. Do not infer guarantees beyond the code.

Expected result. The hook exposes records, administrations, loaded, error, reload, save, signAdministration, canPrescribe and canAdminister. Shared carer functions are retained because the existing table/page interface requires them; the lab does not implement a separate carer workflow.

Verification before continuing. In the hook, find the single point after PATCH where setRecords executes. Explain what the caller receives on 409 and why finally is needed. Confirm the endpoint string never references NEXT_PUBLIC_API_URL. Full type checking waits for the missing components/page restored next.

(8) Build editable rows with explicit save feedback

Rebuild the shared GP table and its signature component dependency. Each row needs a local draft for typing, a version for concurrency, and explicit state for unsaved, saving, saved and failed writes.

Commands / implementation. In terminal A:

POWERSHELL

```
Copy-Item -LiteralPath "$ref\components\prescriber-signature.tsx" -Destination
'..\components\prescriber-signature.tsx'
Copy-Item -LiteralPath "$ref\components\gp-medication-table.tsx" -Destination
'..\components\gp-medication-table.tsx'
```

In MedicineRows, retain the existing imports and JSX and recreate this state/persistence section exactly:

Source: frontend/components/gp-medication-table.tsx (lines 16-63; verbatim excerpt).

TSX

```
const initial = record?.prescription ?? { ...initialPrescription(medication), refer_prn: kind === 'prn' };
const [draft, setDraft] = useState(initial);
const draftRef = useRef(initial);
const version = useRef(record?.version ?? 0);
const dirty = useRef(false);
const savingRef = useRef(false);
const [saving, setSaving] = useState(false);
const [notice, setNotice] = useState('');
const [error, setError] = useState('');
const [signing, setSigning] = useState(false);
useEffect(() => {
  if (!dirty.current && !savingRef.current && record && record.version > version.current) {
    version.current = record.version; draftRef.current = record.prescription;
    setDraft(record.prescription);
  }
}, [record]);
useEffect(() => {
  const warn = (event: BeforeUnloadEvent) => { if (dirty.current || savingRef.current) { event.preventDefault();
    event.returnValue = ''; } };
  window.addEventListener('beforeunload', warn);
  return () => window.removeEventListener('beforeunload', warn);
}, []);

const persist = async (prescription: Prescription) => {
  if (!editable || savingRef.current) throw new Error('Please wait for the current save to finish. ');
  validatePrescription(prescription);
  savingRef.current = true; setSaving(true); setError('');
  try {
    const saved = await save(id, prescription, version.current);
    version.current = saved.version; draftRef.current = saved.prescription;
    dirty.current = false; setDraft(saved.prescription); setNotice('Saved');
  } finally { savingRef.current = false; setSaving(false); }
};

const saveFields = async () => {
  if (!dirty.current || savingRef.current) return;
  try { await persist(draftRef.current); }
  catch (e) { setError(e instanceof Error ? e.message : 'Changes were not saved. '); }
};

const change = <K extends PrescriptionField | 'refer_prn' | 'times'>(field: K, value: Prescription[K]) => {
  if (!editable) return;
  draftRef.current = amendPrescription(draftRef.current, field, value);
  dirty.current = true; setDraft(draftRef.current); setNotice('Unsaved changes'); setError('');
};

const input = (field: PrescriptionField, label: string, type = 'text'): ReactNode => <label
  className={`chart-cell-field chart-field-${field}`}>
  <span>{label}</span>
  <input aria-label={` ${label} - ${id}` } type={type} value={draft[field]} maxLength={200} readOnly={!editable}
  disabled={saving}
  onChange={(event) => change(field, event.target.value)} onBlur={() => void saveFields()}>
```

TSX / CONTINUED

```
min={field === 'end_date' ? draft.start_date || undefined : undefined} />
</label>;
```

Key lines. The row starts with a persisted prescription if one exists, otherwise a fixture-derived default. `dirty` protects an in-progress local draft from refreshes. `version.current` records the server version the user edited. `persist` validates, awaits save, accepts the server's canonical record, clears `dirty`, and only then sets `Saved`. A failed save leaves the draft available and is caught by `saveFields` for display. `change` marks a draft unsaved and invalidates its signature. Text/time fields persist on blur; Refer PRN persists directly from its change handler.

Contract. Parent-to-row props are `record`, `editable`, `save`, and `context/history` inputs. The save callback is `(id, prescription, version) => Promise<ChartRecord>`. A rejection must propagate, not be turned into apparent success. Server capability, selected role and load errors determine editability. Version and `dirty` state belong to the medicine row; network-loaded records belong to the hook.

Why / failure modes / pattern. This is **controlled form state** with a **local draft and server-confirmed commit**. It is not an optimistic success update. Replacing the draft on every poll can erase unsaved typing. Never updating `version.current` after a successful save makes the next write conflict with your own previous write. Setting `Saved` before awaiting the promise misrepresents a failed persistence attempt.

The current protection has limits: `beforeunload` covers browser unload, not every in-app navigation. Switching chart sections unmounts rows, and there is no complete navigation-blocking workflow for dirty drafts. Wait for `Saved` before changing patient, role, home or chart type. The hook's single saving lock also means a save in one row can briefly reject a save in another; use the displayed retry feedback.

Expected result. Rows preserve uncommitted input on a failed save, show a readable error and offer **Retry save / Reload page**. On a version conflict, capture the intended change, reload current state, review it and re-enter the amendment; retrying the unchanged stale version cannot resolve 409.

Verification before continuing. Identify the JSX input's `onChange` and `onBlur`, and connect each to `change`, `saveFields`, `persist` and `chart.save`. Explain why both `draftRef` and `draft` exist: immediate event sequencing and reactive rendering. Do not test a save until step (9) restores the page.

(9) Assemble the GP page and patient context

Restore the page that selects a patient, invokes the hook and passes capabilities into the table. Keep the existing role provider, shell and people pages as shared prerequisites; they supply navigation and context, not authentication.

Commands / implementation. In terminal A:

POWERSHELL

```
Copy-Item -LiteralPath "$ref\app\medication-charts\page.tsx" -Destination '.\app\medication-charts\page.tsx'
npx tsc --noEmit --incremental false
```

The page's context boundary and hook invocation are:

Source: `frontend/app/medication-charts/page.tsx` (lines 23-43; verbatim excerpt).

TSX

```
export default function MedicationChartsPage() {
  const { homeId, role } = useRole();
  const params = useSearchParams();
  return <MedicationChartsContent key={` ${homeId}:${role}:${params.get('client')} ?? ''}` />;
}

function MedicationChartsContent() {
  const searchParams = useSearchParams();
  const router = useRouter();
  const { homeId, role } = useRole();
  const home = getHome(homeId);
```

TSX / CONTINUED

```
const requestedClient = searchParams.get('client');
const requestedMedication = searchParams.get('med');
const clientId = home.clients.some((item) => item.id === requestedClient) ? requestedClient! :
  home.clients[0].id;
const [section, setSection] = useState<'routine' | 'prn'>('routine');
const [pending, setPending] = useState<PendingSignature | null>(null);
const [confirmed, setConfirmed] = useState(false);
const [signed, setSigned] = useState<Record<string, string>>({});
const [savedNotice, setSavedNotice] = useState('');
const client = home.clients.find((item) => item.id === clientId) ?? home.clients[0];
const chart = usePrescriberChart(home.id, client.id, role);
```

The exact GP table call is:

Source: frontend/app/medication-charts/page.tsx (lines 87-87; verbatim excerpt).

TSX

```
{role === 'gp' || role === 'carer' ? <GpMedicationTable key={section} client={client}
  records={chart.records} editable={chart.canPrescribe} loaded={chart.loaded} error={chart.error}
  reload={chart.reload} save={chart.save} kind={section} canAdminister={chart.canAdminister}
  administrations={chart.administrations} onSign={chart.signAdministration} role={role} /> : <section
  className="medication-order-stack">
```

The shared root layout is:

Source: frontend/app/layout.tsx (lines 24-26; verbatim excerpt).

TSX

```
export default function RootLayout({ children }: Readonly<{ children: React.ReactNode }>) {
  return <html lang="en-AU"><body><RoleProvider>{children}</RoleProvider></body></html>;
}
```

Key lines. The keyed MedicationChartsContent remounts when home, role or client query changes. This resets hook/row state across context changes. The client selection only accepts an ID in the selected home's fixture list, otherwise it selects that home's first client. GpMedicationTable key={section} resets row state between routine and PRN views. editable={chart.canPrescribe} uses the server-derived capability rather than only the page's cosmetic canPrescribe flag.

Contract. /medication-charts?client=james-miller is a page URL; /api/prescriber-chart?home=banksia-house&client=james-miller is its data request. UI state changes the former, then the hook derives the latter. The RoleProvider stores only adac4care-role and adac4care-home in localStorage. The role type here is lowercase 'gp' from data/demo/types.ts; the unrelated older lib/auth.ts uses uppercase roles and does not include GP.

Why / failure modes / pattern. This is **container/presentation composition** and **context-scoped state reset**. Forgetting the remount boundary can retain the previous client's record map. Confusing the page's role flag with authorization permits misleading controls. Hiding a navigation item does not protect a server endpoint. getClient on the profile page also falls back to the first demo client for an unknown ID; it is not an authenticated patient lookup.

Expected result. TypeScript exits successfully without output. Restart terminal B with `npm run dev -- --port 3000`. Open James Miller's chart as General practitioner: eight regular order blocks, six time lines per block, editable prescriber fields, and disabled administration cells. GP navigation offers People, Medication charts, Clinical reports and Messages; it does not expose the Overview navigation item. There is no distinct implemented GP dashboard route.

Verification before continuing. Reload and confirm GET is 200. Change patient to Lina Chen and back; inspect the query string and GET context each time. Confirm a GP cannot activate an administration signing cell. The source page includes other-role branches because it is shared; leave those outside your rebuild focus.

(10) Trace the drawn signature through persistence

Study the component restored in step (8) and connect its drawing events to the row's save callback. The goal is to recognize the difference between drawing on a canvas, confirming a network save and retaining a historical authorization snapshot.

Commands / implementation. Recreate the following functions within `PrescriberSignatureDialog` from its complete reference file. These exact source ranges depend on the refs/state declared at the top of that component.

Source: `frontend/components/prescriber-signature.tsx` (lines 36-55; verbatim excerpt).

TSX

```
const save = async () => {
  const typed = !autoSave && entryMode === 'type';
  if (pointer.current !== null || savingRef.current || (!typed && !strokes.current.length)) return;
  if (!autoSave && !confirmed) return;
  const revision = drawingRevision.current;
  savingRef.current = true;
  setStatus('saving');
  setError('');
  try {
    if (typed) {
      const signature = validateAdministrationSignature([], initials);
      await onSave(signature.strokes, signature.initials);
    } else await onSave(strokes.current.map((stroke) => stroke.map((point) => [...point])));
    savedRevision.current = revision;
    if (mounted.current) setStatus(drawingRevision.current === revision ? 'saved' : 'drawing');
    if (!autoSave) onClose();
  } catch (e) {
    if (mounted.current) { setStatus('error'); setError(e instanceof Error ? e.message : 'Signature was not saved. '); }
  } finally { savingRef.current = false; }
};
```

Source: `frontend/components/prescriber-signature.tsx` (lines 82-103; verbatim excerpt).

TSX

```
const end = (event: PointerEvent<HTMLCanvasElement>) => {
  if (pointer.current !== event.pointerId) return;
  pointer.current = null;
  if (strokes.current.at(-1)?.length < 2) strokes.current.pop();
  if (strokes.current.length) {
    drawingRevision.current += 1;
    if (autoSave) timer.current = setTimeout(() => void save(), 1000);
  } else setStatus('blank');
};

const close = async () => {
  if (savingRef.current || pointer.current !== null) return;
  if (drawingRevision.current > savedRevision.current && strokes.current.length) { if (timer.current)
    clearTimeout(timer.current); await save(); }
  if (savedRevision.current === drawingRevision.current) onClose();
};

const clear = () => {
  if (timer.current) clearTimeout(timer.current);
  strokes.current = []; canvas.current?.getContext('2d')?.clearRect(0, 0, 1200, 440);
  setInitials('');
  // Clearing the pad does not erase the last saved signature on the chart.
  drawingRevision.current = savedRevision.current;
  setStatus('blank'); setError('');
};
```

The GP row passes a freshly built signature to persistence here:

Source: frontend/components/gp-medication-table.tsx (lines 106-109; verbatim excerpt).

TSX

```
{signing && editable ? <tr className="signature-dialog-row"><td colSpan={kind === 'prn' ? 20 : 4 +
  days.length}><PrescriberSignatureDialog open onClose={() => setSigning(false)} medicine={draft.name}
  onSave={async (strokes) => {
    const next = { ...draftRef.current, prescriber_signature: { strokes, signedAt: '', signedBy: '' } };
    await persist(next);
  }} /></td></tr> : null}
```

Key lines. Strokes are arrays of normalized $[x,y]$ points in $[0,1]$. The validator accepts 1-100 strokes, at least two points per stroke, some actual movement, and at most 12,000 total points. Canvas pixel dimensions affect drawing presentation, not persisted point coordinates. SVG displays the stored points later; the request contains structured arrays rather than executable SVG markup.

The save is triggered after a one-second pause or by Done if the current drawing has not been saved. `savedRevision` tracks which local drawing revision was acknowledged, preventing Done from treating an unsaved drawing as complete.

Clear box resets the pad; it does not delete the previously persisted chart signature. GP mode uses drawn signatures; typed initials are a shared administration feature.

Contract. The dialog calls `onSave(strokes)` and expects a promise that rejects on failure. The row wraps strokes as `{strokes, signedAt: '', signedBy: ''}` inside a full prescription and sends the PATCH. The route fills those two metadata values from the server. A non-null signature requires completed prescription fields; validation failures are 400, and stale versions are 409. The dialog displays success only after the callback resolves.

Why / failure modes / pattern. This is **debounced persistence** with **server-authoritative metadata**. Calling `onSave` without awaiting it would show false success. Storing only a canvas reference would lose the image on reload. Accepting client-supplied `signedBy` would allow a caller to claim another author. A drawing image alone does not establish identity; the authorization helper remains essential.

Expected result. A completed row can open a blank signature dialog. A drawn signature becomes a stored revision and reappears after reload. A later prescription amendment removes the current signature but leaves the earlier version in D1.

Verification before continuing. On a blank **regular row 2** (not reserved row 8), enter name Test medicine, then dose 10 mg, route Oral, frequency Daily, start date 2026-09-01, and time 08:00. Enter the name first because every blur validates it; wait for Saved after each field. Click Prescriber Signature, draw a visibly moving stroke, wait for the saved message and click Done. Reload and verify the drawing persists. Record the returned version in Network; it need not be 1 because each field blur can create a revision.

(11) Implement scheduling, PRN and readable history without inventing clinical logic

Inspect how the same GP table handles routine times, PRN instructions and date bounds. Preserve administration history when a GP changes times, and identify which dosage or interaction rules the repository actually lacks.

Commands / implementation. Inspect these exact rule functions from the file rebuilt in step (2):

Source: frontend/lib/prescriber-chart.ts (lines 35-37; verbatim excerpt).

TYPESCRIPT

```
export function validateDate(value: string) {
  return /^d{4}-d{2}-d{2}$/.test(value) && Number.isFinite(Date.parse(value)) && new
    Date(value).toISOString().slice(0, 10) === value;
}
```

Source: frontend/lib/prescriber-chart.ts (lines 91-93; verbatim excerpt).

TYPESCRIPT

```
export function administrationOrderReady(p: Prescription) {
  return Boolean(p.name.trim() && p.dose.trim() && p.route.trim() && p.frequency.trim() &&
    validDate(p.start_date));
}
```

The table derives routine cell identity and dates as follows:

Source: frontend/components/gp-medication-table.tsx (lines 65-69; verbatim excerpt).

TSX

```
const seed = medication?.administrations.filter((row) => row.date.startsWith(month)) ?? [];
const events = storedAdministrations.filter((event) => event.medicationId === id);
const today = new Intl.DateTimeFormat('en-CA', { timeZone: 'Australia/Sydney', year: 'numeric', month:
  '2-digit', day: '2-digit' }).format(new Date());
const maySignDate = (date: string) => canAdminister && administrationOrderReady(draft) && medication?.status !==
  'ceased' && date <= today && date >= draft.start_date && (!draft.end_date || date <= draft.end_date);
const signatureCell = (key: string, description: string, enabled: boolean, existing?: AdministrationRecord,
  initial?: string) => <AdministrationSignatureCell key={key} medicationId={id} cellKey={key}
  version={version.current} prescription={draft} record={existing} initial={initial} canSign={enabled}
  description={` ${clientName}, ${draft.name || 'empty medicine'}, ${description}` } onSign={onSign} />;
```

Source: frontend/components/gp-medication-table.tsx (lines 73-76; verbatim excerpt).

TSX

```
const date = `${month}-${String(day).padStart(2, '0')}`;
const key = `routine:${date}:${draft.times[line]}`;
const previous = seed.find((row) => row.date === date && row.time === draft.times[line]);
return signatureCell(key + (draft.times[line] ? ' : ' : ':blank-$line'), `${date} at ${draft.times[line] ||
  'no prescribed time'}`, Boolean(draft.times[line]) && maySignDate(date), events.find((event) =>
  event.cellKey === key), previous?.initial);
```

POWERSHELL

```
Get-Content -LiteralPath '.\app\api\chart-administrations\route.ts'
Get-Content -LiteralPath '.\components\gp-medication-table.tsx' | Select-String
'historical|historySources|max_dose|prn:'
```

Key lines. Routine cells are identified by routine:YYYY-MM-DD:HH:mm. PRN cells use prn:YYYY-MM:slot, with slots 0-23 in four groups of six. The server assigns PRN date/time in Australia/Sydney; the GP edits instructions and max_dose, not those administration timestamps. Changing times does not relocate an old event: the table shows events at removed times in a separate history section. Both fixture and persisted administrations are considered.

Contract. Routine and PRN prescriptions use the same PATCH endpoint and DTO. New row IDs determine chart kind (regular-slot-N or prn-slot-N); ticking refer_prn is a saved field, not a conversion of the medication ID or fixture type. Existing fixture quantity is used when an administration is written; new orders fall back to the saved dose string. GP cannot POST an administration even if the prescription was authored by that GP.

Why / failure modes / pattern. This is **stable historical identity** and **separation of prescribing from administration**. Re-keying previous events after a schedule change would misstate their original times. Using the browser clock for the saved PRN timestamp would let clients disagree. Keeping a max-dose string is only storage/display: the code does not sum doses over 24 hours, calculate dosing, detect drug interactions or automatically cross-check allergies. Allergy text is shown from fixtures; there is no drug knowledge service or notification scheduler.

The active prescription DTO has no status enum or DELETE operation. A stop date bounds administration; it does not remove the row or necessarily update active-order counts elsewhere. Existing fixture status === 'ceased' is checked by administration logic. Those mechanisms are different from Django's DRAFT/ACTIVE/CEASED status field.

Expected result. Up to six unique routine times can be saved. Duplicate times or stop-before-start values fail validation. PRN supports eight order blocks and free-text instruction/MAX fields. Historical events at different times appear separately. Other screens and tab counts based only on fixture arrays do not automatically include newly saved orders.

Verification before continuing. On the fictional row 2 created in step (10), change time to 09:00, wait for Saved and verify the signature is cleared. On the existing fixture row, expand **Previous administration records at other times** and inspect its original dates/times. Switch to PRN, inspect MAX/Instruction fields, and confirm administration cells remain read-only as GP. Do not interpret absence of an alert as validation of a clinical choice.

(12) Trace the separate Django API and explain the missing bridge

Inspect the GP-related Django models, serializers, permissions and routes already present in the scaffold. This comparison teaches how to recognize a second integration contract without claiming that it powers the current chart.

Commands / implementation. In terminal A:

```
POWERSHELL
Set-Location -LiteralPath "$lab\backend"
Get-Content -LiteralPath '.\config\urls.py'
Get-Content -LiteralPath '.\apps\medications\models.py'
Get-Content -LiteralPath '.\apps\accounts\middleware.py'
```

The actual medication permission, serializer and viewset are:

Source: backend/apps/accounts/permissions.py (lines 1-10; verbatim excerpt).

```
PYTHON
from rest_framework.permissions import BasePermission, SAFE_METHODS

class MedicationOrderPermission(BasePermission):
    """Everyone in the authorised care team may read; only a GP may prescribe."""

    def has_permission(self, request, view):
        if request.method in SAFE_METHODS:
            return request.user.is_authenticated
        return getattr(request.user, "role", None) in {"GP", "ADMIN"}
```

Source: backend/apps/medications/serializers.py (lines 1-11; complete file).

```
PYTHON
from rest_framework import serializers
from .models import MedicationOrder

class MedicationOrderSerializer(serializers.ModelSerializer):
    created_by = serializers.HiddenField(default=serializers.CurrentUserDefault())

    class Meta:
        model = MedicationOrder
        fields = "__all__"
        read_only_fields = ["created_at", "updated_at"]
```

Source: backend/apps/medications/views.py (lines 1-13; complete file).

```
PYTHON
from rest_framework.viewsets import ModelViewSet
from .models import MedicationOrder
from .serializers import MedicationOrderSerializer
from apps.accounts.permissions import MedicationOrderPermission

class MedicationOrderViewSet(ModelViewSet):
    serializer_class = MedicationOrderSerializer
    permission_classes = [MedicationOrderPermission]

    def get_queryset(self):
```

PYTHON / CONTINUED

```

organisation_id = getattr(self.request.user, "organisation_id", None)
return MedicationOrder.objects.select_related("patient",
    "patient__home").filter(patient__home__organisation_id=organisation_id) if organisation_id else
    MedicationOrder.objects.none()

```

The frontend's unused Django request helper is:

Source: frontend/lib/api-client.ts (lines 1-19; complete file).

TYPESCRIPT

```

const API_URL = process.env.NEXT_PUBLIC_API_URL ?? 'http://localhost:8000/api';

export class ApiError extends Error {
  constructor(public status: number, message: string) {
    super(message);
  }
}

export async function apiRequest<T>(path: string, init: RequestInit = {}): Promise<T> {
  const headers = new Headers(init.headers);
  if (!headers.has('Content-Type')) headers.set('Content-Type', 'application/json');
  const response = await fetch(`${API_URL}${path}`, {
    ...init,
    credentials: 'include',
    headers,
  });
  if (!response.ok) throw new ApiError(response.status, await response.text());
  return response.json() as Promise<T>;
}

```

Key lines. DRF's router registers medication-orders; ModelViewSet generates list/create/retrieve/update/destroy routes. MedicationOrderPermission allows authenticated reads and GP/ADMIN writes. get_queryset scopes existing objects by the user's organisation. created_by is a hidden current-user field. The model uses UUID IDs and fields such as patient, drug, one scheduled_time, starts_on, ends_on, and status, instead of the chart's string IDs and six-slot time array. Model save() calls full_clean().

Contract. /api/medication-orders/ has a standard DRF model-object contract: GET list returns an unpaginated JSON array; POST success 201; detail GET/PUT/PATCH success 200; DELETE success normally 204, subject to protected relations/model failures. A GP must have an authenticated Django session and request.session['mfa_verified'] for API access. mfa_enabled=True on User is not the session verification flag. Validation normally uses field-error objects or {detail:...}, not the D1 route's {error:...}.

The browser helper sets credentials: 'include', but no current GP component imports/calls apiRequest. The backend only configures SessionAuthentication. The checked-in URLs expose no dedicated GP session-login, JWT, token refresh, or MFA-completion endpoint. Azure identity variable names exist in the environment example, but do not constitute an implemented auth integration. Do not invent credentials or turn off MFA to pretend this workflow is complete.

Why / failure modes / pattern. Django uses **router/viewset/serializer/ORM layering, session-authenticated REST and tenant-scoped reads**. Moving to it would require a deliberate **API adapter / anti-corruption layer**: map client/medicine IDs, DTO fields, scheduling semantics, version history and errors; then implement the intended session/MFA/CSRF flow. A base URL swap alone cannot do that. The helper sends no CSRF token, and credentialed cross-origin CORS support is not fully configured in the inspected settings. Django model ValidationError is not explicitly translated by these serializers, so some model-level failures can become server errors rather than a tidy 400.

There are further implementation gaps: queryset filtering does not itself validate that a submitted patient foreign key belongs to the user's organisation; MedicationOrderSerializer does not add that relation check. This API updates medication orders in place and exposes DELETE, so it does not inherit D1's revision guarantees. AuditAccessMiddleware records authenticated API accesses separately, including response status, but it does not synchronize the frontend charts. No GP medication controller/service layer beyond the viewset/model is present.

Expected result. Django's unauthenticated request remains 403. The source shows GP/ADMIN medication write permission and a separate relational model. No new D1 row appears from reading or testing Django.

Verification before continuing. In **terminal C**, stop Django with Ctrl+C, run its existing tests and restart. This terminal is still in the lab backend directory and retains the three environment assignments from step (0):

POWERSHELL

```
& .\venv\Scripts\python.exe -m pytest -q
& .\venv\Scripts\python.exe manage.py runserver 127.0.0.1:8000 --noreload
```

Expected: four tests pass. These are model/permission tests, not a completed browser login/MFA flow. In Network on the GP chart, confirm no calls to port 8000. State the five things an adapter would need beyond changing an environment variable: identity, IDs, field shapes, concurrency/history and error handling.

(13) Distinguish persistent GP features from reports and notification demos

Inspect the GP's surrounding screens so your reconstruction has the same honest scope as the repository. A visible success notice does not imply an HTTP request or a durable database write.

Commands / implementation. In terminal A:

POWERSHELL

```
Set-Location -LiteralPath "$lab\frontend"
Get-Content -LiteralPath '.\app\reports\page.tsx' | Select-String 'canCreate|setNotice|fetch'
Get-Content -LiteralPath '.\app\messages\page.tsx' | Select-String 'setNotice|fetch'
Get-Content -LiteralPath '.\components\app-shell.tsx' | Select-String 'notification|count: 3'
```

The exact GP report capability and notification button are:

Source: frontend/app/reports/page.tsx (lines 24-24; verbatim excerpt).

TSX

```
const canCreate = role === 'rn' || role === 'gp';
```

Source: frontend/components/app-shell.tsx (lines 143-143; verbatim excerpt).

TSX

```
<Button aria-label="Open messages and notifications" className="notification-button" onClick={() =>
  router.push('/messages')} size="icon" variant="ghost"><Bell /><span /></Button>
```

Key lines. Reports derive from each fixture client's reports; GP can open the compose dialog, but its final button only closes it and sets Clinical report saved as a demo record. Messages likewise sets Communication saved as a demo record. Neither handler sends a request or appends a durable record. Report acknowledgements use component state. The bell navigates to /messages; the menu count is hard-coded as 3.

Contract. These UI actions have no implemented backend request/response/status contract. They are **local presentation state / prototype interactions**. The separate Django /api/messages/ model endpoint exists, but these screens do not call it. The chart save does not dispatch an email, push notification, background alert, report or message as a side effect.

Why / failure modes / pattern. This is the distinction between **ephemeral UI state** and **durable application state**. If a success notice is treated as proof of persistence, the user may believe a care-team instruction was stored when only text changed on screen. Adding real notifications would require a durable trigger and delivery system; neither is silently supplied by polling or by rendering a Bell icon.

Expected result. Existing fixture reports and messages are readable. A compose action can display demo feedback, but no new persisted feed record survives reload. People/profile/Overview data remains fixture-derived; new chart changes are not propagated into all those screens. The manifest exists, but no implemented offline prescription queue or service-worker registration was found in the inspected app code.

Verification before continuing. In the lab browser, clear Network, open Clinical reports, click Add GP instruction, enter a fictional title and click Sign & add report. Confirm no report POST occurs and the feed does not gain a durable record

after reload. This is a deliberate gap observation, not a requested implementation of notifications or other roles.

(14) Verify one complete GP workflow over HTTP and in the UI

Run the supplied local-only client against a fresh reserved row, then verify a further amendment through the actual browser. Together these checks connect request validation, server identity, D1 history, hook state and visible save feedback.

Commands / implementation. Keep terminal B running. In terminal A:

POWERSHELL

```
Set-Location -LiteralPath $repo
node output/gp-lab/Test-GpWorkflow.mjs http://localhost:3000
```

This is the exact request object from the new lab verification file; the name and dose are fictional test values:

Source: output/gp-lab/Test-GpWorkflow.mjs (lines 27-33; verbatim excerpt).

JAVASCRIPT

```
const prescription = {
  name: 'Test medicine', dose: '10 mg', route: 'Oral', frequency: 'Daily',
  start_date: '2026-09-01', end_date: '', refer_prn: false,
  times: ['08:00', '', '', '', '', ''], instructions: 'Fictional software test only.',
  max_dose: '', prescriber_signature: null,
};
const body = { medicationId, version: 0, prescription };
```

Read its complete implementation before running if you want to predict the assertions:

POWERSHELL

```
Get-Content -LiteralPath "$repo\output\gp-lab\Test-GpWorkflow.mjs"
```

Key lines. The script refuses a non-local hostname and refuses to reuse row 8 if it already has history. It sends wrong-role, wrong-origin and unknown-field requests before the successful write. It creates version 1, retries the stale version (409), signs version 2, rejects a dose change carrying the old drawing (400), then saves an unsigned amended version 3 and reads it back. It also handles a plain-text runtime rejection instead of assuming every failure is JSON.

Contract. The expected successful row is regular-slot-8, version 3, dose 20 mg, signature null. Versions 1 and 2 remain 10 mg; version 2 retains its signed drawing. The server records Local GP evaluation as actor in local development. This proves local persistence and attribution rules, not the authentication of a deployed clinical identity.

Why / failure modes / pattern. This is a **vertical integration test** through HTTP plus a **browser acceptance check**. Unit tests alone cannot prove routing, bindings and migrations agree. Reading after a write checks persisted state; visually checking the browser confirms that the frontend understands the returned contract. A stale client must be rejected without overwriting a newer order.

Expected result. The script prints PASS: local HTTP read... and describes the expected three revisions. Any failing assertion stops it; do not rerun over existing row history. If an earlier attempt created row 8, continue by inspecting that state or create a new isolated exercise directory instead of deleting its history.

Verification before continuing. Open James Miller's chart as General practitioner and scroll to regular order 8. Verify Test medicine, 20 mg and Awaiting prescriber signature. Change the dose to 25 mg, press Tab, wait for **Saved**, then reload. It must still show 25 mg. That browser action should append version 4, not update version 3. In terminal A:

POWERSHELL

```
Set-Location -LiteralPath "$lab\frontend"
npx wrangler d1 execute DB --local --config wrangler.lab.json --command "SELECT medication_id, version, actor,
  json_extract(prescription, '$.dose') AS dose FROM prescription_revisions WHERE medication_id='regular-slot-8'
ORDER BY version"
```

Expected doses by version: 1 -> 10 mg; 2 -> 10 mg; 3 -> 20 mg; 4 -> 25 mg. Check version 2's non-null signature separately by inspecting prescription. You have now verified a create-sign-amend-read workflow and an actual blur-save-reload UI path.

(15) Exercise conflicts, recover failures and close the lab

Finish by observing failure behavior rather than only the happy path. Keep every experiment in the isolated local scaffold and retain the database so revision history remains inspectable.

Commands / implementation. In terminal A, run the existing final checks after your reconstruction:

```
POWERSHELL
Set-Location -LiteralPath "$lab\frontend"
node --experimental-strip-types --test tests/prescriber-chart.test.ts
node tests/chart-administrations.mjs
npx tsc --noEmit --incremental false
```

Key lines. The first command checks rules and hosted-mode capability decisions; the second exercises production-mode handlers against isolated SQL; the third checks that page props, hook return types and DTO declarations agree. No npm test script exists. npm run build is the real production build command if you need a build artifact, and npm run start uses wrangler dev --config dist/server/wrangler.json; it requires a build first and is not the same as local DEV evaluation. Production build/start were not part of this lab's completed author validation.

Contract. In two browser tabs, dirty the same row in each before either saves. Save tab A first; then blur tab B's stale draft. Tab B should receive 409 and retain its draft/error. Refreshing a clean tab or focusing it lets GET reconcile newer records. For a recoverable failure, stop the frontend server, change a field in an already loaded chart and blur; the row must not report Saved. Restart the server, retry, and read back. A missing-table 503, network failure and 409 require different recovery actions.

Why / failure modes / pattern. These are **failure-injection checks**, **conflict detection** and **explicit recovery**. A blind retry may be appropriate for a transient network failure only after checking whether the first save actually committed; it is not the right response to a confirmed version conflict. There is no automatic merge or offline queue. A dirty second tab intentionally resists overwriting by background GET, so copying the intended edit aside before a full reload avoids losing it.

Expected result. All 10 frontend domain tests, the route SQL harness and TypeScript pass. A stale write does not increment the database version; a failed connection does not produce Saved. Do not infer authorization from a disabled button alone: the production-mode test supplies unauthorized requests directly.

Verification / cleanup. Keep a short evidence note: tested home/client/row, initial/final versions, Network status, SQL readback and observed UI message. Stop terminals B and C with Ctrl+C, then close the lab tabs. No database deletion is required; the scaffold and its reference remain under work/gp-user-lab for comparison. In terminal A, return to the original workspace:

```
POWERSHELL
Set-Location -LiteralPath $repo
```

Before finishing, explain these three invariants in your own words: only the server grants writes; every accepted edit uses the expected version; a Saved indicator follows server acknowledgement. To apply this to another stack, identify its equivalents of the DTO, authenticated actor, resource lookup, atomic write and UI reconciliation point before writing the form.

Appendix

A. GP-related endpoint quick reference

All chart routes below use the frontend origin. Chart GET/PATCH require `home` and `client` query parameters. Their handler-level errors use `{error: string}` and no-store responses; hosting/runtime errors can differ.

Method + route	Purpose / success	Authentication and authorization
GET <code>/api/prescriber-chart</code>	Latest saved revisions + administrations + capability flags; 200.	Hosted authenticated viewer; local evaluation permitted. No per-client grant table is implemented.
PATCH <code>/api/prescriber-chart</code>	Create or amend a row through full snapshot + expected version; 200.	Hosted identity in <code>ADAC_GP_EMAILS</code> , <code>X-Chart-Role: gp</code> , matching Origin. Local bypass still checks role and Origin.
POST <code>/api/chart-administrations</code>	Shared prerequisite that produces the history GP reads; 201 new, 200 identical retry.	Hosted <code>ADAC_CARER_EMAILS</code> , carer mode, matching Origin, current valid order. GP cannot write. No separate GET; chart GET returns history.

`/api/chart-administrations` errors: 403 identity/role/origin; 404 missing order/context; 400 malformed signature/cell/date/confirmation; 409 ceased, incomplete, changed or already signed order/cell; 413 oversize; 503 storage failure. An identical retry is idempotent only when it reaches the duplicate-record check: earlier current-order/version checks can still reject a retry after an order changes.

The following endpoints belong to **Django**, normally on port 8000, and are not used by the active GP chart. All require an authenticated session; GP API access also encounters custom MFA middleware. Detail identifiers are UUIDs. Lists have no configured pagination. Framework OPTIONS/HEAD behavior is not expanded below.

Method + route	GP purpose / standard success	Role and scope
GET <code>/api/homes/</code> , <code>/api/homes/{id}/</code>	Read homes; 200.	Authenticated; organisation-filtered. GP cannot create/edit/delete profiles.
GET <code>/api/clients/</code> , <code>/api/clients/{id}/</code>	Read patients; 200.	Authenticated; organisation-filtered. GP cannot create/edit/delete profiles.
GET <code>/api/medication-orders/</code> , <code>/api/medication-orders/{id}/</code>	Read orders; 200.	Authenticated; organisation-filtered querysets.
POST <code>/api/medication-orders/</code>	Create model object; 201.	GP or ADMIN; foreign-key tenant validation gap described in step (12).
PUT / PATCH <code>/api/medication-orders/{id}/</code>	Replace/amend model fields; 200.	GP or ADMIN; existing object constrained by queryset. No D1 revision contract.
DELETE <code>/api/medication-orders/{id}/</code>	Exposed destroy action; normally 204 if permitted by relations.	GP or ADMIN. Protected relations may prevent deletion.
GET <code>/api/administrations/</code> , <code>/api/administrations/{id}/</code>	Read administration events; 200.	GP is read-only; role permission denies GP writes.

Method + route	GP purpose / standard success	Role and scope
GET / POST /api/messages/	Read/create ChartMessage; 200 / 201.	Authenticated care messaging; GP included. Read queryset organisation-filtered.
GET / PUT / PATCH / DELETE /api/messages/{id}/	Retrieve/update/delete message; 200 / 200 / 200 / 204.	Authenticated; existing-object queryset filtering. No extra sender-ownership rule implemented.
GET /api/schema/, /api/docs/	OpenAPI schema / Swagger UI.	Schema view permissions default separately; custom MFA middleware still applies to authenticated privileged users. Not a GP login route.

Django common failures include 403 unauthenticated/session/MFA/CSRF/permission, 404 out-of-scope or missing detail, and 400 serializer field errors. Model/database failures are not all normalized; do not promise every error is JSON or 400. `config/urls.py` also exposes Django admin; it does not supply an application GP login/MFA completion flow.

B. Database tables and domain fields

Active D1 table / source	Important columns / meaning
prescription_revisions	home_id, client_id, medication_id, version composite PK; prescription JSON text; server actor, saved_at. Latest projection uses MAX(version).
administration_signatures	Composite PK home/client/medication/cell_key; order_version, date, time, qty, signature JSON, server actor/timestamp. GP reads these.
Imported demo JSON	Homes, clients, baseline medications, allergies, reports and messages. No D1 patient/GP/user table is consulted by chart routes.
medication_records	Separate shared audit feature: id, author_id, author_email, created_at, payload. Not the GP prescription store; excluded from the rebuild except migration/scaffold prerequisites.

Prescription JSON fields: name, dose, route, frequency, start_date, end_date, instructions, max_dose, refer_prn, times[6], prescriber_signature. A signature is {strokes, signedAt, signedBy}; the broader shared type permits administration initials. GP validation requires a drawing. There is no prescription DELETE handler, arbitrary medicine count beyond eight slots per type, GP status-transition endpoint or dosage-calculation service.

Separate Django table	GP-relevant fields
accounts_user	Inherited user/session identity; role (GP uppercase), organisation_id, mfa_enabled. Actual MFA check reads session key mfa_verified.
clients_organisation	UUID id, name, ABN; parent of homes/users.
clients_home	UUID id, organisation_id, name, suburb, state, active.
clients_client	UUID id, home_id, names, date of birth, allergies JSON, NDIS number, notes, active, timestamps.

Separate Django table	GP-relevant fields
medications_medicationorder	UUID id, patient_id, drug, dose, route, scheduled_time, documentation_instructions, reason, expected_response, prn, prn_protocol, prescriber, starts_on, ends_on, status, created_by_id, timestamps.
administration_administrationevent	Order/staff references, confirmation fields and observed outcome; shared read-only context for GP.
messaging_chartmessage	UUID id, patient_id, optional medication_order_id, sender_id, recipient_type, subject/body, created_at/read_at.
audit_auditevent	Actor, action, resource type/id, request path/id, metadata, occurred_at. ORM instance save/delete protects existing events; this is separate from D1 revisions.

C. Environment variables and runtime bindings

Name	Layer / actual effect
DB	Cloudflare Workers D1 binding, not a browser env string. .openai/hosting.json names it; Vite config creates the local binding.
ADAC_GP_EMAILS	Worker runtime string; comma-separated allowed GP emails. Empty denies hosted writes; local DEV bypasses the allowlist.
ADAC_CARER_EMAILS	Shared Worker allowlist for administration signing. Included only because chart capabilities/history share that implementation.
import.meta.env.DEV	Vite build/runtime constant, not a .env variable to set. It selects local evaluation behavior.
NEXT_PUBLIC_API_URL	Read only by lib/api-client.ts; example http://localhost:8000/api. Does not change the active GP relative endpoint.
NEXT_PUBLIC_SITE_URL	Used by app/layout.tsx for metadata URL, default http://localhost:3000. Not an API connection.
NEXT_PUBLIC_APP_ENV	Present in frontend example; no active GP consumer found.
DJANGO_SETTINGS_MODULE	config.settings.dev for lab. manage.py defaults to it; pytest.ini also specifies it.
DJANGO_SECRET_KEY	Server secret; local example only. Existing real secrets are not copied.
DJANGO_DEBUG	Set true before loading settings for local HTTP. Drives security flags in base settings.
DJANGO_ALLOWED_HOSTS	Example localhost,127.0.0.1. Validates Django Host headers.
DATABASE_URL	Django database only. Lab sqlite:///gp-lab.sqlite3; copied example points at local PostgreSQL.
CORS_ALLOWED_ORIGINS	Django browser-origin allowlist, example localhost:3000. Does not authenticate users or supply CSRF tokens; not relevant to same-origin GP requests.
AZURE_AD_B2C_TENANT, AZURE_AD_B2C_CLIENT_ID	Backend example placeholders; no implemented GP identity flow found consuming them.

Name	Layer / actual effect
WRANGLER_SEND_METRICS, WRANGLER_LOG_PATH	Lab tooling controls, not frontend/backend business-data configuration.
MINIFLARE_REGISTRY_PATH, WRANGLER_WRITE_LOGS	Vite config sets project-local runtime/tool logging defaults. Not clinical data storage or auth controls.

The repo does not include a confirmed hosted evaluator allowlist or an authenticated GP provisioning workflow. Configure real hosted identities through the appropriate runtime/identity administration separately; entering an email in a client-side role selector is not provisioning. No JWT secret, refresh URL or invented DB connection string is required for the implemented local GP path.

D. Troubleshooting by evidence

Observation	Interpretation / next check
GP chart GET 503	Inspect D1 binding and local migrations in the same frontend directory. Django migrations are a different database.
GET 404 Client not found	Check the home/client pair; valid example is banksia-house + james-miller.
PATCH 403	Check role header, Origin, hosted identity and GP allowlist. Local dropdown changes presentation, not hosted provisioning.
Plain-text Forbidden / JSON parse error	Runtime may reject Origin before route JSON code. Inspect Network response status/body instead of assuming data . error exists.
PATCH 400 on partial {dose}	Send the full Prescription shape and expected version. Inspect exact field/date/time error.
PATCH 409	Read current revision and review changes; preserve your intended edit before reloading. Do not keep retrying an old version.
Row appears but SQL count is 0	Baseline fixture data is rendered without saved revisions.
Changed prescription vanishes after navigation	Check whether Saved appeared and PATCH succeeded; dirty draft navigation is not fully guarded.
Signature disappears after field edit	Intended invalidation. Previous signed revision remains; re-sign the changed values.
New order missing from People/Overview counts	Those screens/counts use fixture arrays, not the chart's persisted projection.
Django redirects local HTTP to HTTPS	Ensure DJANGO_DEBUG=true before base settings load; dev.py alone does not recompute all flags.
Django authenticated GP still gets 403 MFA	The session's mfa_verified flag is absent; no completed application MFA flow is supplied here.
Node test/Vite/Wrangler spawn EPERM	In the author's sandbox this required approved child-process execution; it was not a GP code defect. Do not change permission logic to fix tool restrictions.

E. Source evidence and completed author validation

The document uses the supplied PDFs only for pedagogical structure: header, objective, setup, numbered laboratory instructions beginning at (0), explanatory code exercises and appendix. All subject matter, commands and examples here originate from this repository or clearly identified lab helpers.

Application code fences were extracted from the source snapshot, not reconstructed from README claims. The companion `source-manifest.json` records SHA-256 hashes for referenced application files, and `source/` contains complete source files for offline comparison. That bundle excludes credentials, installed dependencies, existing databases and the sample PDFs. Line references identify the original snapshot; reconstruction destinations are under `work/gp-user-lab`.

Executed successfully during preparation: isolated scaffold creation; D1 migrations 0000 and 0001; frontend development server; HTTP 200 page and GP GET; Django migrations/check/dev server and its unauthenticated 403 response; all 10 frontend rule/access tests; existing production-mode route/SQLite harness; TypeScript without emit; all 4 Django tests; local HTTP create/sign/conflict/amend/readback verification; SQL readback of revisions 1-3; browser GP selection and a dose blur showing Saved. Final browser reload and database verification are recorded in the accompanying validation note.

Limits: existing installed frontend dependencies and an existing Python virtual environment were reused for author validation of the isolated copy; fresh `npm ci` / `pip` downloads were not rerun. They are the verified manifest-based learner install commands. Production build, hosted deployment, real identity provisioning, authenticated Django GP/MFA browser flow and clinical validation were not performed. The lesson explicitly distinguishes expected learner observations from these completed checks. No changes to the original application source were required.